ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA

BÁO CÁO BÀI TẬP LỚN
ĐẠI SỐ TUYẾN TÍNH

ĐỀ TÀI:
CHU TRÌNH EULER VÀ CHU TRÌNH HAMILTON

GVHD: Huỳnh Thái Duy Phương

Thành phố Hồ Chí Minh – 11/2023

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA

BÁO CÁO BÀI TẬP LỚN
ĐẠI SỐ TUYẾN TÍNH

Nhóm: 9 – Nhóm trưởng: Huỳnh Trung Kiên
Email: kien.huynhtrungkien@hcmut.edu.vn

STT	Họ và tên	MSSV	Đóng góp
1	Huỳnh Đạt Đức	2310772	0%
2	Nguyễn Thanh Hậu	2310933	100%
3	Đỗ Nhật Khanh	2311489	0%
4	Huỳnh Trung Kiên	2311729	100%
5	Trịnh Đình Thái	2313106	100%
6	Lê Thanh Trọng	2313644	100%
7	Văn Quý Tuân	2313724	100%

MỤC LỤC

1. MA TRẬN KỀ ..... - 4 -
1.1. Định nghĩa ..... - 4 -
1.1.1. Đồ thị vô hướng ..... - 4 -
1.1.2. Đồ thị có hướng ..... - 4 -
1.2. Biểu diễn ..... - 4 -
1.2.1. Đồ thị vô hướng ..... - 4 -
1.2.2. Đồ thị có hướng ..... - 5 -
1.3. Nhận xét ..... - 5 -
2. ĐƯỜNG ĐI, CHU TRÌNH VÀ ĐỒ THỊ LIÊN THÔNG ..... - 6 -
2.1. Đường đi ..... - 6 -
2.1.1. Định nghĩa ..... - 6 -
2.1.2. Ví dụ ..... - 7 -
2.2. Liên thông ..... - 7 -
2.2.1. Định nghĩa 1 ..... - 7 -
2.2.2. Định nghĩa 2 ..... - 8 -
2.2.3. Định nghĩa 3 ..... - 8 -
2.2.4. Định nghĩa 4 ..... - 9 -
3. CHU TRÌNH EULER ..... - 9 -
3.1. Định nghĩa ..... - 9 -
3.2. Định lý Euler ..... - 10 -
3.2.1. Định lý 1 ..... - 10 -
3.2.2. Định lý 2 ..... - 10 -
3.2.3. Định lý 3 ..... - 11 -
4. CHU TRÌNH HAMLTON ..... - 11 -
4.1. Định nghĩa ..... - 11 -
4.2. Định lý Hamlton ..... - 11 -
4.2.1 Định lý 1 ..... - 11 -
4.2.2. Định lý 2 ..... - 13 -
4.2.3. Định lý 3 ..... - 13 -
5. THUẬT TOÁN TÌM CHU TRÌNH HAMILTON (BACKTRACKING) ..... - 13 -
5.1. Định nghĩa ..... - 13 -
5.2. Nguyên tắc hoạt động ..... - 14 -
6. CHƯƠNG TRÌNH TÌM CHU TRÌNH EULER, HAMILTON ..... - 15 -
7. KẾT LUẬN ..... - 20 -

1. MA TRẬN KỀ

Ma trận kề (adjacency matrix) là ma trận vuông dùng để biểu diễn đồ thị hữu hạn. Các phần tử của ma trận cho biết các cặp định có liền kề nhau hay không trong đồ thị. Mỗi đồ thị chỉ có duy nhất một ma trận kề, mỗi đồ thị khác nhau có mỗi ma trận kề của riêng chúng.

1.1. Định nghĩa

- Xét đồ thị có tập đỉnh U = { u_1 , …, u_n }, ma trận kệ của nó là ma trận vuông $A_{n\,x\,n}$ sao cho chần tử A_{ij} của nó là 1 khi có một cạnh từ đỉnh u_i đến đỉnh u_i và bằng 0 khi không có cạnh. Các phần tử đường chéo của ma trận đều bằng 0, vì các cạnh từ một đỉnh đến chính nó (vòng lặp) không được phép trong đồ thị đơn giản.
- A= (A_{ij}) = 1 nếu có cạnh nối giữa u_i đến u_j
- A= (A_{ij}) = 0 nếu không có cạnh nối giữa u_i đến u_j

1.1.1. Đồ thị vô hướng

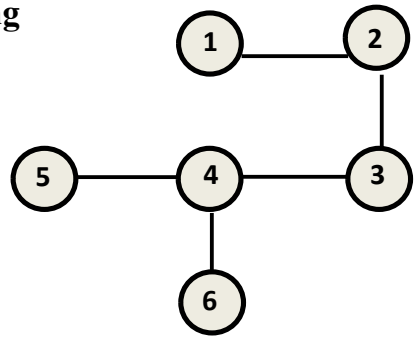
- Đồ thị vô hướng (tức là tất cả các cạnh của nó là hai chiều) thì ma trận kề là đối xứng.

1.1.2. Đồ thị có hướng

- Ma trận kề của đồ thị có hướng có thể không đối xứng. Người ta có thể định nghĩa ma trận kề của đồ thị có hướng sao cho phần tử khác 0 biểu thị cạnh từ *i* đến *j* và không có chiều ngược lại.

1.2. Biểu diễn

1.2.1. Đồ thị vô hướng



Đồ thị V

Gọi A là ma trận kề của đồ thị V ta có:

1 và 2 có 1 cạnh nối → $A_{12} = A_{21} = 1$

2 và 3 có 1 cạnh nối → $A_{23} = A_{32} = 1$

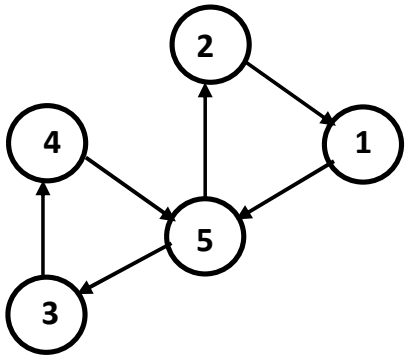
3 và 4 có 1 cạnh nối → $A_{34} = A_{43} = 1$

4 và 5 có 1 cạnh nối → $A_{45} = A_{54} = 1$

4 và 6 có 1 cạnh nối → $A_{46} = A_{64} = 1$

	1	2	3	4	5	6
1	**0**	1	0	0	0	0
2	1	**0**	1	0	0	0
3	0	1	**0**	1	0	0
4	0	0	1	**0**	1	1
5	0	0	0	1	**0**	0
6	0	0	0	1	0	**0**

1.2.2. Đồ thị có hướng



Đồ thị C

Gọi B là ma trận kề của đồ thị C

1 và 2 có 1 cạnh nối, 2 đi vào 1→ $B_{21} = 1$

5 và 2 có 1 cạnh nối, 5 đi vào 2→ $B_{52} = 1$

5 và 1 có 1 cạnh nối, 1 đi vào 5→ $B_{15} = 1$

5 và 4 có 1 cạnh nối. 4 đi vào 5→ $B_{45} = 1$

5 và 3 có 1 cạnh nối, 5 đi vào 3→ $B_{53} = 1$

3 và 4 có 1 cạnh nối, 3 đi vào 4→ $B_{34} = 1$

	1	2	3	4	5
1	**0**	0	0	0	0
2	1	**0**	0	0	0
3	0	0	**0**	0	0
4	0	0	1	**0**	0
5	1	1	1	1	**0**

1.3. Nhận xét

- Ở cách biểu diễn ma trận này, ma trận sẽ không biểu diễn được đồ thị có cạnh song song.

- Ưu điểm lớn nhất của phương pháp biểu diễn đồ thị bằng ma trận kề là để trả lời câu hỏi: Hai đỉnh u,v có kề nhau trên đồ thị hay không, chúng ta chỉ phải thực hiện một phép so sánh.
- Nhược điểm lớn nhất của phương pháp này là: không phụ thuộc vào số cạnh của đồ thị, ta luôn phải sử dụng n^2 đơn vị bộ nhớ để lưu trữ ma trận kề của nó.

2. ĐƯỜNG ĐI, CHU TRÌNH VÀ ĐỒ THỊ LIÊN THÔNG

2.1. Đường đi

2.1.1. Định nghĩa

- Đường đi độ dài n từ đỉnh u đến đỉnh v, trong đó n là số nguyên dương, trên đồ thị vô hướng G = <V, E> là dãy:

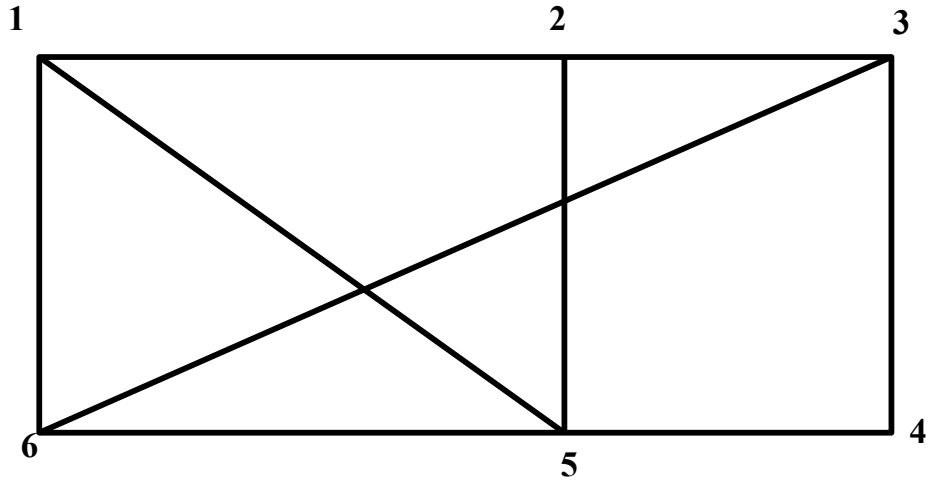
$$x_0, x_1, x_2, \ldots, x_{n-1}, x_n$$

  trong đó $u = x_0$, $v = x_n$, (x_i, x_{i+1}) E, i = 0, 1, 2,…, n-1.
- Đường đi nói trên còn có thể biểu diễn dưới dạng dãy các cạnh:

$$(x_0, x_1), (x_1, x_2), \ldots, (x_{n-1}, x_n)$$

- Đỉnh u gọi là đỉnh đầu, còn đỉnh v gọi là đỉnh cuối của đường đi.
- Đường đi có đỉnh đầu trùng với đỉnh cuối (tức là u = v) được gọi là chu trình.
- Đường đi hay chu trình được gọi là đơn nếu như không có cạnh nào bị lặp lại.
- Đường đi hay chu trình được gọi là sơ cấp nếu như không có đỉnh nào bị lặp lại trên đường đi.

2.1.2. Ví dụ



1→2→3→4 là đường đi đơn có độ dài bằng 5

2→3→4→5→2 là chu trình có độ dài bằng 6

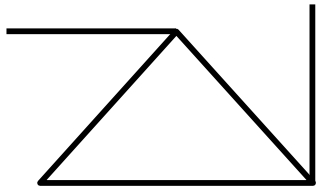
1→2→4→5 không phải là đường đi do 2→4 không phải là cạnh

2.2. Liên thông

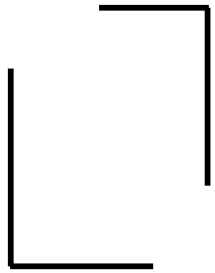
2.2.1. Định nghĩa 1

- Đồ thị vô hướng G = (V, E) được gọi là liên thông nếu luôn tìm được đường đi giữa hai đỉnh bất kỳ của nó.

Ví dụ:



Đồ thị liên thông G



Đồ thị không liên thông K

2.2.2. Định nghĩa 2

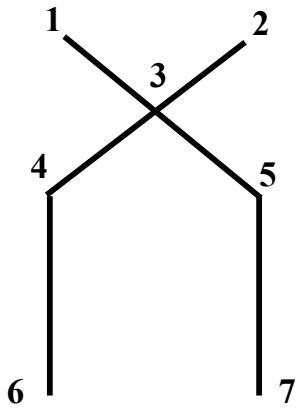
- Ta có đồ thị G = (V,E). Đồ thị H = (W,F) là đồ thị con của G nếu và chỉ nếu W⊆V và F⊆E.
- Trong trường hợp đồ thị là không liên thông, nó sẽ rã ra thành một số đồ thị con liên thông không có đỉnh chung. Những đồ thị con liên thông như vậy ta sẽ gọi là các thành phần liên thông của đồ thị.

Ví dụ: Đồ thị K trong hình 1 gồm 2 thành phần liên thông.

2.2.3. Định nghĩa 3

- Đỉnh v được gọi là đỉnh rẽ nhánh nếu việc loại bỏ v cùng với các cạnh liên thuộc với nó khỏi đồ thị làm tăng số thành phần liên thông của đồ thị.
- Cạnh e được gọi là cầu nếu việc loại bỏ nó khỏi đồ thị làm tăng số thành phần liên thông của đồ thị.

Ví dụ:



Đồ thị G

Trong đó:

3, 4, 5 là đỉnh rẽ nhánh.

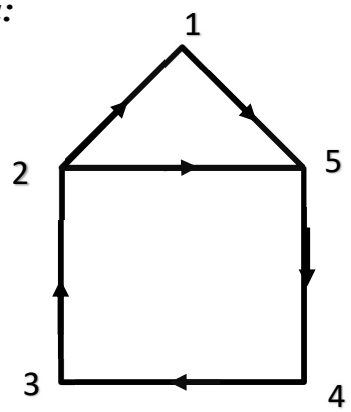
Cạnh (3,4), cạnh (3,5) là cầu.

2.2.4. Định nghĩa 4

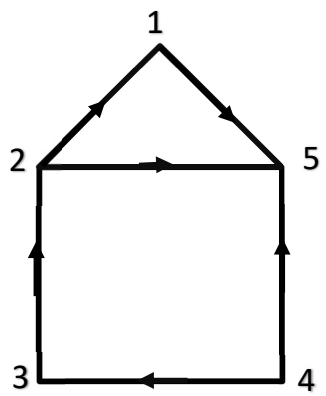
Đồ thị có hướng G = (V, A) được gọi là liên thông mạnh nếu luôn tìm được đường đi giữa hai đỉnh bất kỳ của nó.

Đồ thị có hướng G = (V, A) được gọi là liên thông yếu nếu đồ thị vô hướng tương ứng với nó là vô hướng liên thông.

Ví dụ:



Đồ thị liên thông mạnh



Đồ thị liên thông yếu

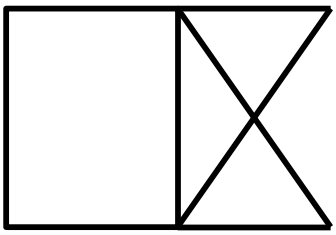
3. CHU TRÌNH EULER

3.1. Định nghĩa

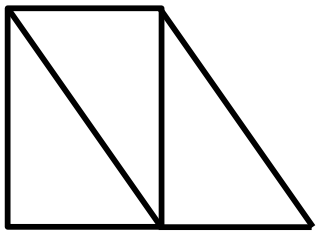
Giả sử G là đơn đồ thị (đa đồ thị) vô hướng (có hướng), ta có:

- Chu trình Euler trong G là chu trình đơn đi qua tất cả các cạnh của đồ thị.
- Nếu G có chu trình Euler thì G được gọi là đồ thị Euler.
- Đường đi Euler trong G là đường đi đơn qua tất cả các cạnh của đồ thị. Nếu G có đường đi Euler thì G được gọi là nửa đồ thị Euler.

Ví dụ:



Đồ thị Euler



Đồ thị nửa Euler

3.2. Định lý Euler

3.2.1. Định lý 1

Đồ thị vô hướng, liên thông G = (V,E) có chu trình Euler khi và chỉ khi mọi đỉnh của G đều có bậc chẵn.

- **Bổ đề:** Nếu bậc của mỗi đỉnh của đồ thị G không nhỏ hơn 2 thì G chứa chu trình.
- ***Chứng minh:***

  Xét đường đi từ một đỉnh v_0 bất kì: $v_0 \rightarrow v_1 \rightarrow v_2 \rightarrow \ldots$Vì bậc của tất cả các đỉnh đều lớn hơn hoặc bằng 2 nên nếu v_i là một đỉnh mới của đường đi này thì khi không có đường đi vào v_i thì nhất định của một đường đi ra khỏi v_i. Vì số đỉnh của đồ thị là hữu hạn nên đường đi sẽ quay về đỉnh cũ v_k bất kì. Khi đó ta có ít nhất một chu trình.

  Điều kiện cần: Nếu G có chu trình Euler thì mọi đỉnh của G đều có bậc chẵn.

  Gán trọng số cho các đỉnh là 0. Xét chu trình Euler của đồ thị, tăng trọng số của đỉnh lên 2 khi đi qua đỉnh đó. Do chu trình sau khi đi qua tất cả các đỉnh sẽ quay lại điểm xuất phát nên trọng số (bậc) của tất cả các đỉnh sẽ là chẵn.

  Điều kiện đủ: Nếu mọi đỉnh của G đều có bậc chẵn thì G có chu trình Euler.
- **Hệ quả:** Đồ thị vô hướng liên thông G là nửa Euler khi và chỉ khi nó có không quá 2 đỉnh bậc lẻ.

3.2.2. Định lý 2

Đồ thị vô hướng, liên thông G = (V,E) có đường đi Euler mà không có chu trình Euler khi và chỉ khi G có đúng hai đỉnh bậc lẻ.

3.2.3. Định lý 3

G có hướng liên thông mạnh G là đồ thị Euler khi và chỉ khi mọi đỉnh của nó đều có bán bậc ra bằng bán bậc vào.

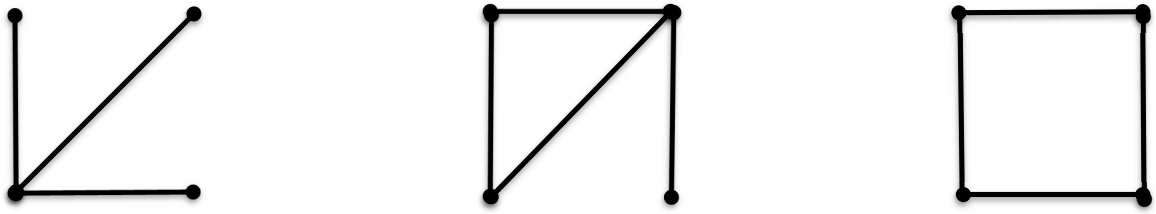
$$\mathbf{deg^+(v) = deg^-(v), \forall\, v \in V}$$

4. CHU TRÌNH HAMLTON

4.1. Định nghĩa

- Đường đi qua tất cả các đỉnh của đồ thị mỗi đỉnh đúng một lần được gọi là đường đi Hamilton.
- Chu trình bắt đầu từ một đỉnh v nào đó qua tất cả các đỉnh còn lại mỗi đỉnh đúng một lần rồi quay trở về v được gọi là chu trình Hamilton.
- Đồ thị G được gọi là đồ thị Hamilton nếu nó chứa chu trình Hamilton và gọi là đồ thị nửa Hamilton nếu nó có đường đi Hamilton.

Ví dụ: Trong hình: Đồ thị G3 là Hamilton, G2 là nửa Hamilton, G1 không là nửa Hamilton.



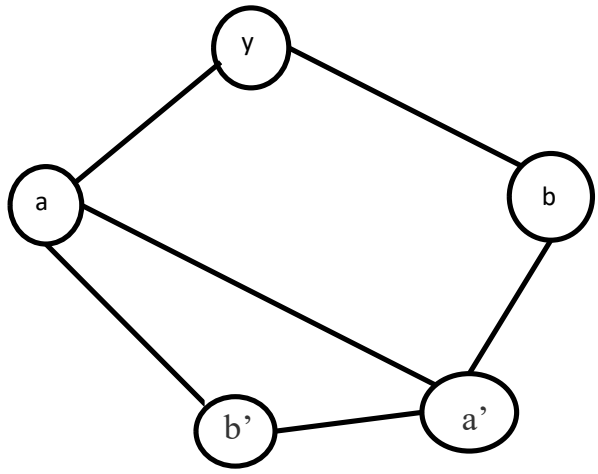
G1 **G2** **G3**

4.2. Định lý Hamlton

4.2.1 Định lý 1

Đơn đồ thị vô hướng G với n > 2 đỉnh, mỗi đỉnh có bậc không nhỏ hơn n/2 là đồ thị Hamilton.

Ví dụ:



- ***Chứng minh:***

Định lý được chứng minh bằng phản chứng. Giả sử G không có chu trình Hamilton. Ta thêm vào G một số đỉnh mới và nối mỗi đỉnh mới này với mọi đỉnh của G, ta được đồ thị G'. Giả sử k (>0) là số tối thiểu các đỉnh cần thiết để G' chứa một chu trình Hamilton. Như vậy, G' có n+k đỉnh.

Gọi P là chu trình Hamilton ayb ...a trong G', trong đó a và b là các đỉnh của G, còn y là một trong các đỉnh mới. Khi đó b không kề với a, vì nếu trái lại thì ta có thể bỏ đỉnh y và được chu trình ab ...a, mâu thuẫn với giả thiết về tính chất nhỏ nhất của k.

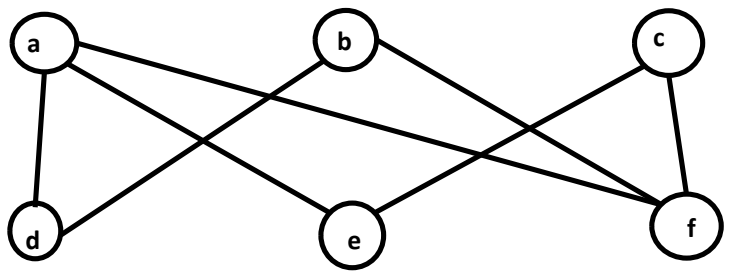
Ngoài ra, nếu a' là một đỉnh kề nào đó của a (khác với y) và b' là đỉnh nối tiếp ngay a' trong chu trình P thì b' không thể là đỉnh kề với b, vì nếu trái lại thì ta có thể thay P bởi chu trình aa' ...bb' ... a, trong đó không có y, mâu thuẫn với giả thiết về tính chất nhỏ nhất của k.

Như vậy, với mỗi đỉnh kề với a, ta có một đỉnh không kề với b, tức là số đỉnh không kề với b không thể ít hơn số đỉnh kề với a (số đỉnh kề với a không nhỏ hơn n/2 +k). Mặt khác, theo giả thiết số đỉnh kề với b cũng không nhỏ hơn n/2+k. Vì không có đỉnh nào vừa kề với b lại vừa không kề với b, nên số đỉnh của G' không ít hơn 2(n/2+k)=n+2k, mâu thuẫn với giả thiết là số đỉnh của G' bằng n+k (k>0). Định lý được chứng minh.

4.2.2. Định lý 2

Nếu G là đồ thị phân đôi với hai tập đỉnh là V1, V2 có số đỉnh cùng bằng n ($n \geq 2$) và bậc của mỗi đỉnh lớn hơn n/2 thì G là một đồ thị Hamilton.

Ví dụ:



Đồ thị phân đôi này có bậc của mỗi mỗi đỉnh bằng 2 hoặc bằng 3 (>3/2) nên theo định lí 2 nó là đồ thị Hamilton.

4.2.3. Định lý 3

Giả sử G là đồ có hướng liên thông với n đỉnh. Nếu $deg^+(v) \geq n/2$, $deg^-(v) \geq n/2$, $\forall v$ thì G là Hamilton.

5. THUẬT TOÁN TÌM CHU TRÌNH HAMILTON (BACKTRACKING)

5.1. Định nghĩa

- Thuật toán backtrack là một phương pháp giải quyết các bài toán tìm kiếm, liệt kê hoặc xây dựng các tập hợp con theo cách lần lượt thử tất cả các lựa chọn có thể, và từ bỏ những lựa chọn dẫn đến kết quả không mong muốn. Thuật toán này thường được áp dụng trong các bài toán tối ưu hoặc tìm kiếm có cấu trúc và ràng buộc.

5.2. Nguyên tắc hoạt động

- Nguyên tắc hoạt động của thuật toán backtrack là sử dụng phương pháp quay lui (backtracking) để thử từng lựa chọn có thể và tiếp tục tìm kiếm đến khi đạt được kết quả mong muốn hoặc xác định rằng không có kết quả phù hợp. Khi thử một lựa chọn, thuật toán sẽ tiến hành kiểm tra xem lựa chọn đó có thỏa mãn các ràng buộc hay không. Nếu lựa chọn thỏa mãn, thuật toán sẽ tiếp tục tiến vào pha tiếp theo của bài toán. Ngược lại, nếu lựa chọn không thỏa mãn, thuật toán sẽ quay lại và thử các lựa chọn khác.
- Thuật toán backtrack thường được triển khai bằng đệ quy, với mỗi lần đệ quy tương ứng với việc xem xét một lựa chọn và thử các lựa chọn con tiếp theo. Khi quay lui, thuật toán sẽ trở lại các bước trước đó và thử các lựa chọn khác. Quá trình này tiếp tục cho đến khi tìm được kết quả mong muốn hoặc đã thử tất cả các lựa chọn mà không tìm được kết quả.
- Thuật toán backtrack có thể được áp dụng cho nhiều bài toán khác nhau như xếp lịch, tìm kiếm xâu con, giải các câu đố, tìm kiếm đường đi trong đồ thị, v.v. Tuy nhiên, độ phức tạp thời gian của thuật toán backtrack thường cao và phụ thuộc vào số lượng lựa chọn và cấu trúc của bài toán

Các bước hoạt động của thuật toán:

- **Bước khởi tạo:** Bắt đầu với một trạng thái ban đầu và khởi tạo các biến cần thiết.
- **Kiểm tra điều kiện dừng:** Kiểm tra xem đã đạt được điều kiện dừng của bài toán hay chưa. Điều kiện này có thể là việc tìm thấy giải pháp hoặc đã thử tất cả các lựa chọn mà không tìm được giải pháp.
- **Lựa chọn và kiểm tra ràng buộc:** Chọn một lựa chọn trong không gian tìm kiếm và kiểm tra xem lựa chọn đó có thỏa mãn các ràng buộc hay không. Nếu không thỏa mãn, quay lại và thử lựa chọn khác.

- **Cập nhật trạng thái:** Cập nhật trạng thái của bài toán dựa trên lựa chọn đã chọn. Có thể cập nhật các biến, mảng, hoặc cấu trúc dữ liệu khác để thể hiện giải pháp hiện tại.
- **Đệ quy:** Thực hiện đệ quy để tiếp tục tìm kiếm các lựa chọn tiếp theo. Quá trình đệ quy sẽ được lặp lại cho đến khi tìm được giải pháp hoặc không còn lựa chọn nào khả thi.
- **Quay lui:** Khi đã thử tất cả các lựa chọn hoặc không thể tiếp tục, quay lại trạng thái trước đó và thử các lựa chọn khác. Quá trình này được gọi là quay lui và cho phép ta khám phá các lựa chọn khác mà chưa được thử.
- **Trả về kết quả:** Trả về giải pháp tìm được khi thuật toán backtrack kết thúc
- Quá trình này tiếp tục cho đến khi tìm thấy giải pháp hoặc đã thử hết tất cả các lựa chọn mà không tìm thấy giải pháp. Thuật toán backtrack tỏ ra hiệu quả khi có thể loại bỏ các nhánh tìm kiếm không cần thiết thông qua các điều kiện ràng buộc và cắt nhánh, giúp giảm không gian tìm kiếm và tăng tốc độ tìm kiếm giải.

6. CHƯƠNG TRÌNH TÌM CHU TRÌNH EULER, HAMILTON

```
def is_valid_Euler(u, v, graph):
    # Kiểm tra xem có cạnh nối u và v hay không
    return graph[u][v] > 0


def eulerian_cycle(graph):
    # Tìm đỉnh xuất phát, bắt đầu từ đỉnh có bậc khác 0
    start_vertex = 0
    for i in range(len(graph)):
        if sum(graph[i]) % 2 != 0:
            start_vertex = i
```

```
            break

    # Stack để lưu chu trình
    stack = [start_vertex]
    euler_circuit = []

    while stack:
        current_vertex = stack[-1]

        # Tìm đỉnh kề đầu tiên của đỉnh hiện tại
        For next_vertex in
range(len(graph[current_vertex])):
            if is_valid_Euler(current_vertex, next_vertex,
graph):
                # Đánh dấu cạnh đã đi qua
                graph[current_vertex][next_vertex] -= 1
                graph[next_vertex][current_vertex] -= 1

                # Thêm đỉnh kề vào chu trình và đẩy vào
stack
                stack.append(next_vertex)
                break

        # Nếu không tìm thấy đỉnh kề, pop đỉnh hiện tại ra
khỏi stack và thêm vào chu trình
        if current_vertex == stack[-1]:
            stack.pop()
            euler_circuit.append(current_vertex)
```

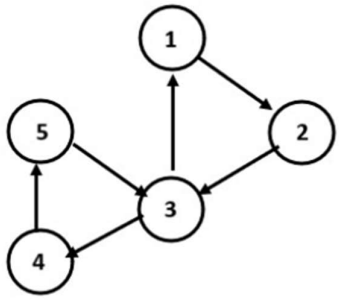
```
    return euler_circuit



def is_valid_Hamilton(v, pos, path, graph):  #nhận diện
đường đi Hamilton
    if graph[path[pos - 1]][v] == 0:
        return False

    if v in path:
        return False

    return True



def hamiltonian_cycle_util(graph, path, pos): #thử tất cả
đường đi
    if pos == len(graph):
        if graph[path[pos - 1]][path[0]] == 1:
            return True
        return False

    for v in range(1, len(graph)):
        if is_valid_Hamilton(v, pos, path, graph):
            path[pos] = v

            if hamiltonian_cycle_util(graph, path, pos +
1):
                return True
```

```
            path[pos] = -1

    return False


def hamiltonian_cycle(graph): #xử lý dữ liệu
    path = [-1] * len(graph)
    path[0] = 0

    if not hamiltonian_cycle_util(graph, path, 1):
        print("Không tìm thấy chu trình Hamilton")
        return False


    print("Chu trình Hamilton: ", end='')

    print(path[0] + 1, end=' ')
    for i in reversed(range(0, len(path))):
        print(path[i] + 1, end=' ')
    print()

    return True

#MAIN:

n = int(input("Nhập số đỉnh: "))
```

```
a = []

tong=0

for i in range(0, n): #nhập ma trận
    row = list(map(int, input(f"Nhập hàng {i + 1} (các số cách nhau bằng dấu cách): ").split()))
    a.append(row)

hamiltonian_cycle(a)

a = eulerian_cycle(a)
if (len(a) <= 1) or (a[0] != a[len(a)-1]): #chuẩn hóa đầu ra
    print('Không tìm thấy chu trình Euler')
else:
    print('Chu Trình Euler:', end=' ')
    for i in range(0, len(a)):
        print(a[i] + 1, end=' ')
    print()
```

Kết quả:

```
Nhập số đỉnh: 5
Nhập hàng 1 (các số cách nhau bằng dấu cách): 0 0 1 0 0
Nhập hàng 2 (các số cách nhau bằng dấu cách): 1 0 0 0 0
Nhập hàng 3 (các số cách nhau bằng dấu cách): 0 1 0 0 1
Nhập hàng 4 (các số cách nhau bằng dấu cách): 0 0 1 0 0
Nhập hàng 5 (các số cách nhau bằng dấu cách): 0 0 0 1 0
Không tìm thấy chu trình Hamilton
Chu Trình Euler: 1 2 3 4 5 3 1
```



Đồ thị minh họa

7. KẾT LUẬN

Chu trình Euler và chu trình Hamilton là hai khái niệm quan trọng trong lý thuyết đồ thị. Chu trình Euler là một chu trình qua tất cả các cạnh của đồ thị một lần và chỉ một lần, trong khi chu trình Hamilton là một chu trình đi qua tất cả các đỉnh của đồ thị một lần và chỉ một lần. Mỗi đỉnh trên chu trình Hamilton có thể được đi qua theo nhiều hướng khác nhau.

Đặc Điểm:

- Chu Trình Euler:

Một đồ thị có chu trình Euler nếu và chỉ nếu mọi đỉnh trong đồ thị có bậc chẵn.

Có thể sử dụng thuật toán Fleury hoặc Hierholzer để tìm chu trình Euler.

- Chu Trình Hamilton:

Vấn đề chu trình Hamilton là một vấn đề NP-đầy đủ, có nghĩa là không có thuật toán hiệu quả đã biết để giải quyết nó trong thời gian đa thức.

Điều kiện đủ để đồ thị có chu trình Hamilton là đồ thị đơn đồng quy và mỗi đỉnh có bậc ít nhất là n/2 (n là số đỉnh của đồ thị).

- Đề Nghị:

Nghiên Cứu Mở Rộng:

Nghiên cứu về thuật toán tìm chu trình Hamilton hiệu quả hơn trong trường hợp đặc biệt hoặc có các ràng buộc cụ thể.

Khám phá tính chất và ứng dụng của chu trình Hamilton trong các lĩnh vực như mạng lưới, truyền thông, hoặc quy hoạch đồ thị.

Áp Dụng Trong Thực Tế:

Áp dụng khái niệm chu trình Euler và chu trình Hamilton vào các vấn đề thực tế như lên lịch sản xuất, quản lý tài nguyên, hoặc định tuyến mạng.

Nghiên cứu cách tích hợp các thuật toán tìm chu trình Euler và Hamilton vào các ứng dụng cụ thể.

Giáo Dục và Tạo Năng Lực:

Tăng cường giáo dục về lý thuyết đồ thị và ứng dụng của chu trình Euler và Hamilton trong chương trình đào tạo cơ bản và cao cấp.

Khuyến khích nghiên cứu sinh và sinh viên thực hiện các dự án liên quan đến chu trình Euler và Hamilton.

Chu trình Euler và chu trình Hamilton đều có ý nghĩa lớn trong lý thuyết đồ thị và có ứng dụng rộng rãi trong nhiều lĩnh vực khác nhau. Việc tiếp tục nghiên cứu và áp dụng chúng sẽ đóng góp vào sự hiểu biết của chúng ta về cấu trúc và tính chất của đồ thị và mở rộng phạm vi ứng dụng của chúng.

TÀI LIỆU THAM KHẢO

Giáo trình Đại số tuyến tính – thầy Đặng Văn Vinh.

https://vi.wikipedia.org/wiki/Đường_đi_Hamilton